Lab 2

Name: Mahdi Davoodzadeh
SN: 405725536
Repository: https://github.com/deep-observability-lab/OS-Tracing.git

1. Objective

In this lab, I investigated how Linux handles memory mapping. I used **Perf** to discover and record kernel tracepoints, and I used **ftrace** to inspect the functions executed inside `do_mmap`.

My main goals were to:

1. Check the requested `kmem`, `netif`, `tcp`, `i2c`, and `mmap` tracepoints.
2. Record relevant memory events with Perf and collect their call stacks.
3. Find and explain the complete observed call stack leading to `do_mmap`.
4. Enable relevant events with ftrace and examine nested `do_mmap` function calls using `function_graph`.

I used the precompiled executable provided with Lab 2. I kept the scripts and raw output files in the `lab2` directory so that the steps and results could be checked later.

2. Test Program and Environment

I performed the experiments on Ubuntu 24.04.5 LTS with kernel `6.17.0-40-generic`, using `perf` 6.17.13, Linux ftrace, and Bash.

I first ran `src/program` without tracing to confirm that it worked correctly. The executable uses memory mapping, writes the character `A` to the mapped memory, and prints the address and the first byte. It completed successfully with exit code 0.

Before starting ftrace, I confirmed that TraceFS was mounted at `/sys/kernel/tracing`, the `function_graph` tracer was available, and the function `do_mmap` was listed in `available_filter_functions`. The original main tracer was `nop`, with no enabled events.

3. Discovering Tracepoints with Perf

I started by listing the kernel tracepoints:

```
sudo perf list tracepoint
```

I checked the event patterns requested in the assignment, then used `scripts/01_perf_event_scan.sh` to run `perf stat` for each event group against the supplied executable. I saved the results separately to make the different groups easier to compare.

Event or group	Observation
`kmem:*`	Several memory-management events were recorded.
`net:netif*`	All selected counters were zero in this run.
`tcp:*`	All selected counters were zero in this run.
`i2c:*`	All selected counters were zero in this run.
`syscalls:sys_enter_mmap`	Available and successfully recorded.
`syscalls:sys_exit_mmap`	Available and successfully recorded.

Some of the measured `kmem` results were:

Event	Perf stat count
`kmem:kmem_cache_alloc`	618
`kmem:kmem_cache_free`	449
`kmem:rss_stat`	276
`kmem:kfree`	125
`kmem:mm_page_alloc`	117
`kmem:mm_page_free`	66
`kmem:kmalloc`	55

These observations were consistent with a program that allocates and accesses memory. The available network, TCP, and I2C tracepoints returned zero counts during their respective test runs, while the memory-event counters recorded activity.

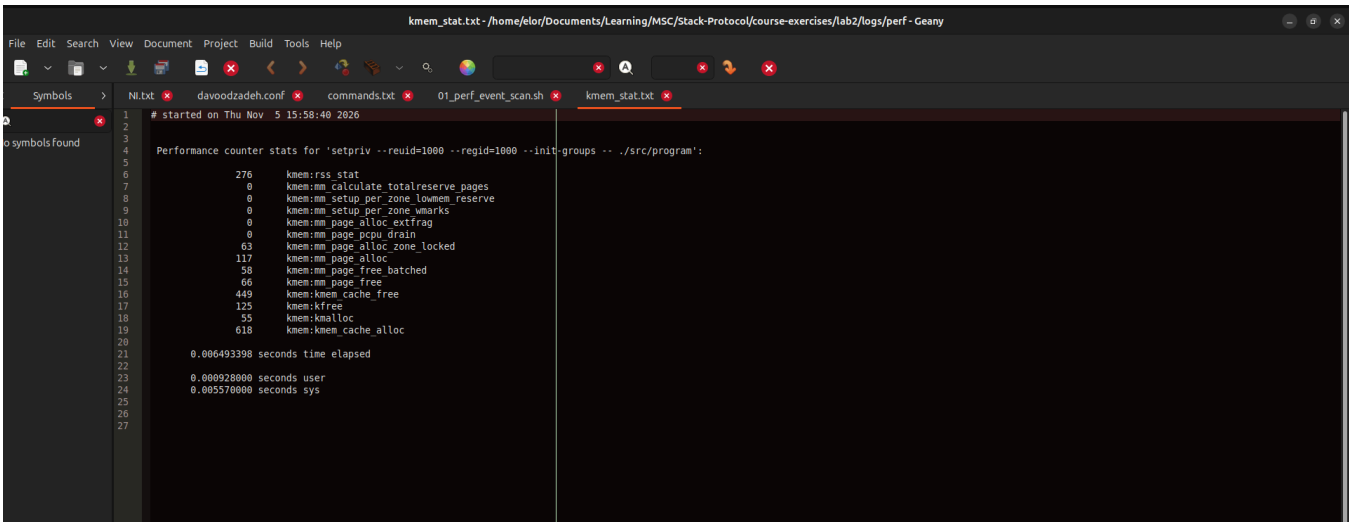


Figure 1 — Perf event discovery: memory counters. The saved statistics show 618 cache allocations, 449 cache frees, 276 RSS events, 117 page allocations, and 66 page frees.

I also used Perf to count `sys_enter_mmap` and `sys_exit_mmap`. This preliminary test recorded **41 entries and 41 exits**, covering the `setpriv` launcher's startup and the program's execution. I separated these scopes in the subsequent call-stack analysis.

4. Recording Memory Events and Call Stacks with Perf

I selected the following events because they were relevant to the supplied program:

```
syscalls:sys_enter_mmap
syscalls:sys_exit_mmap
kmem:kmalloc
kmem:kmem_cache_alloc
kmem:mm_page_alloc
kmem:rss_stat
```

I recorded them using `scripts/02_perf_record_mmap.sh`. The script ran `perf record` with `--call-graph fp`, saved the binary recording, and exported it to text using `perf script`. I launched the test application as my normal user through `setpriv` while Perf collected the privileged trace.

The recording produced **1,204 samples** in `logs/perf/mmap_memory.data`, and the exported text contained **28,009 lines**. I used `scripts/03_perf_inspect_stacks.sh` to count the recorded events.

Recorded event	Count
`syscalls:sys_enter_mmap`	41
`syscalls:sys_exit_mmap`	41
`kmem:kmalloc`	115
`kmem:kmem_cache_alloc`	612
`kmem:mm_page_alloc`	120
`kmem:rss_stat`	275
Total	**1,204**

I also separated the syscall events by the process name shown in Perf:

Process name	mmap entries	mmap exits
`setpriv`	32	32
`program`	9	9

Starting an executable involves memory mapping for its runtime environment and shared libraries. I therefore grouped the 41 syscall pairs by process name and used the call stacks to identify the application’s explicit request from `main`.

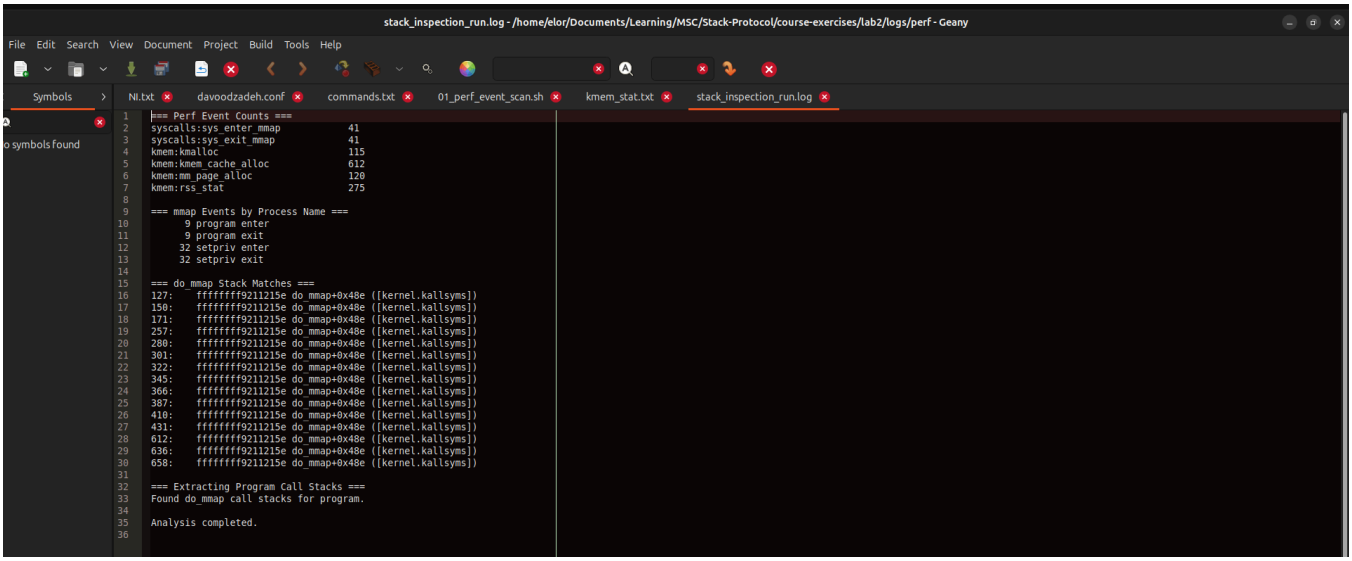


Figure 2 — Perf recording counts and process-name breakdown. The recording contains 1,204 samples, including 32 mmap entry/exit pairs attributed to setpriv and nine pairs attributed to program.

5. Finding the `do_mmap` Call Stack

My initial Perf recording already showed `do_mmap` inside some kernel call stacks. In one case, the stack passed through `load_elf_binary`, `elf_load`, and `vm_mmap`, identifying an executable-loading path.

To capture `do_mmap` more directly, I created a temporary dynamic probe:

```
sudo perf probe --add 'lab2_mmap:entry=do_mmap'
```

I used `scripts/04_perf_probe_do_mmap.sh` to record this probe with call graphs. The recording contained **57 probe hits**: 40 with the process name `setpriv` and 17 with the name `program`. The script removed the temporary probe after recording.

Then I ran `scripts/05_perf_program_mmap.sh` to identify a sample containing the application's `main` function. I found this call sequence, shown below in **calling order** (the original `perf script` stack lists the most recent function first):

```
_start
  -> __libc_start_main
    -> __libc_start_call_main
      -> main+0x3b
        -> __mmap
          -> entry_SYSCALL_64_after_hwframe
            -> do_syscall_64
              -> x64_sys_call
                -> __x64_sys_mmap
                  -> ksys_mmap_pgoff
                    -> do_mmap
```

This is the most important Perf call stack in my experiment. It shows that the application called the C library's `mmap` wrapper, entered the kernel through the system call interface, and reached `do_mmap`, which handles the mapping request.

At the beginning of this stack, `_start` is the executable's entry point. It transfers control to `__libc_start_main` for C runtime initialization, and `__libc_start_call_main` invokes the application's `main`. The `main+0x3b` frame identifies the caller location within `main`. The C library's `__mmap` wrapper

passes the mapping arguments through the x86-64 syscall interface.

I interpreted the intermediate kernel frames individually, using the Linux 6.17 implementation:

Captured frame	Role in the mapping request
`entry_SYSCALL_64_after_hwframe`	An assembly label within the syscall entry routine, after it has constructed the saved return frame on the kernel stack. The following instructions save the remaining register state and call `do_syscall_64` with the register frame and syscall number.
`do_syscall_64`	Handles the C-level syscall entry and exit work, including the entry hooks and dispatch to the appropriate syscall implementation. It also prepares the return to user space after the syscall finishes.
`x64_sys_call`	Dispatches the native 64-bit syscall number to its handler. For this request, it selects `__x64_sys_mmap`.
`__x64_sys_mmap`	The generated x86-64 wrapper extracts the six mmap arguments from the saved registers. The mmap implementation checks that the byte offset is page-aligned, converts it to a page offset, and calls `ksys_mmap_pgoff`.
`ksys_mmap_pgoff`	Prepares the mapping request with an offset expressed in pages. It resolves the file descriptor for file-backed mappings and handles anonymous-mapping options. It passes the request through `vm_mmap_pgoff` to the mapping implementation represented by `do_mmap` in this captured stack.

Finally, `do_mmap` validates the mapping parameters, determines the address range and VMA flags, and calls `mmap_region` to establish or merge the mapping. Sections 7 and 8 show these operations in the recorded function graphs. Together, the frames connect program startup, the application's request, syscall dispatch, and the kernel's virtual-memory management.

The 17 probe hits under the name `program` covered executable loading and application execution. The sample containing both `main` and `__mmap` identified the explicit application request.

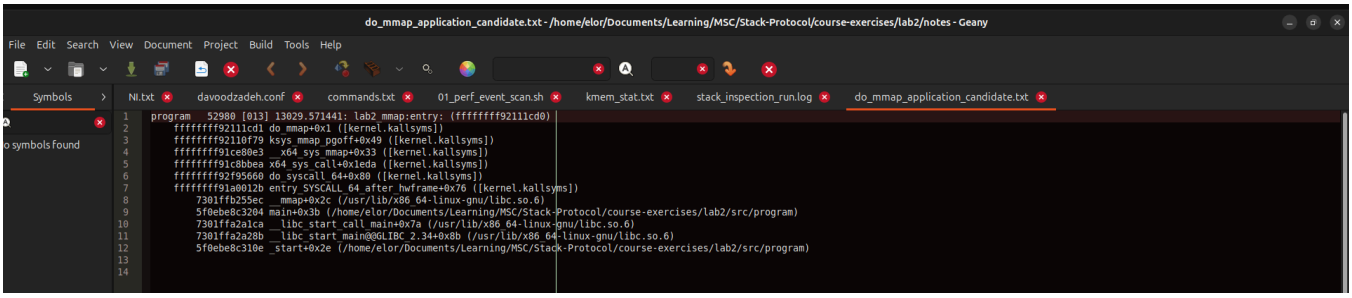


Figure 3 — Perf call stack from main to do_mmap. The captured stack connects the application request to the kernel mapping function; Perf displays the most recent function first.

6. Configuring ftrace and Recording Function Graphs

After the Perf experiments, I used ftrace to examine what happened *inside* `do_mmap`.

I confirmed that `function_graph` was available and that all six selected event tracepoints existed. I used the main TraceFS interface, which provided `set_graph_function`, for the combined Function Graph experiment.

I wrote `scripts/08_ftrace_do_mmap.sh` to perform the following steps:

1. Check the existing main ftrace configuration and save the previous trace buffer as a separate file.
2. Stop tracing temporarily, select `function_graph`, and set `do_mmap` as the graph function.
3. Enable `sys_enter_mmap`, `sys_exit_mmap`, `kmalloc`, `kmem_cache_alloc`, `mm_page_alloc`, and `rss_stat`.
4. Pause the test process before executing the program, configure the PID filters, and resume it.
5. Stop tracing, save the results, disable the selected events, and restore the earlier main tracer settings.

The application exited successfully. I saved the combined trace to `logs/ftrace/08_do_mmap_ftrace.trace`. The file contained **2,274 lines**, including **17 `do_mmap()` entry lines**. Afterward I verified that `current_tracer` was `nop`, `tracing_on` was `1`, and `set_event` was empty, matching the initial configuration.

For the four selected memory events, the combined trace contained:

Event	ftrace count
`kmalloc`	11
`kmem_cache_alloc`	106
`mm_page_alloc`	49
`rss_stat`	127

The Perf and ftrace counts describe separate executions with their respective launch and tracing configurations.

7. Understanding the Functions Inside `do_mmap`

The Function Graph showed the actual nested function calls and their durations. In one recorded mapping path, I observed:

```
do_mmap
  -> __get_unmapped_area
    -> thp_get_unmapped_area
    -> security_mmap_addr
  -> mmap_region
    -> __mmap_region
      -> __mmap_prepare
      -> ext4_file_mmap_prepare
      -> vma_merge_new_range
      -> __mmap_new_vma
        -> vm_area_alloc
      -> vm_stat_account
      -> vma_set_page_prot
```

This excerpt is simplified: the full graph contains more functions and deeper nesting. In this particular path, the kernel created a new virtual memory area (VMA) and performed file-related preparation.

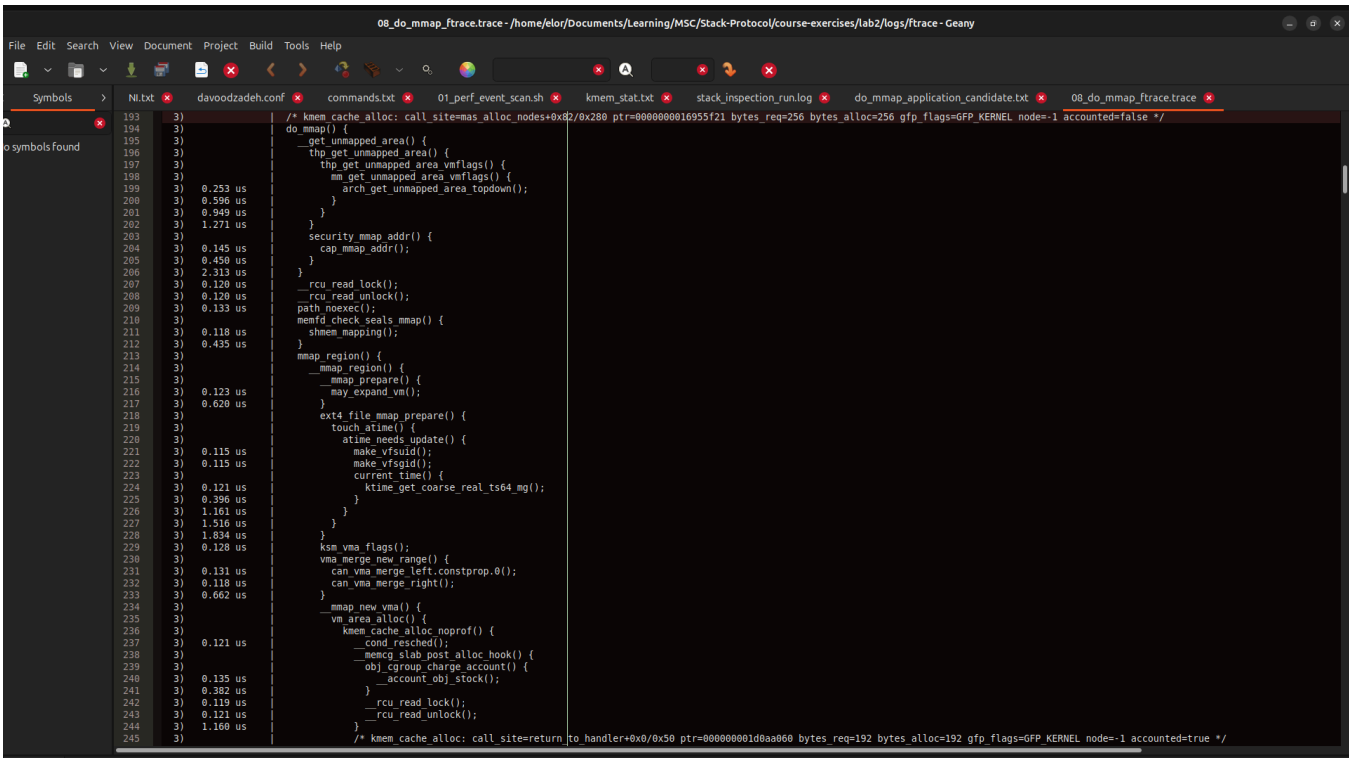


Figure 4 — ftrace Function Graph: the new-VMA path. The graph shows address selection, ext4 file-mapping preparation, and __mmap_new_vma calling vm_area_alloc.

The important functions have the following roles:

Function	What I observed or understood
`do_mmap`	Coordinates the kernel-side memory mapping request.
`__get_unmapped_area`	Finds a suitable unused virtual address range.
`security_mmap_addr`	Performs an address-related security check.
`mmap_region` / `__mmap_region`	Handle the mapping and associated VMA updates.
`__mmap_prepare`	Checks whether the requested mapping can be prepared.
`vma_merge_new_range`	Checks whether the new range can be merged with an existing VMA.
`__mmap_new_vma`	Handles the path that creates a new VMA.
`vma_expand` / `commit_merge`	Handle expansion and merging when a compatible VMA is available.
`vm_stat_account`	Updates virtual memory accounting.
`vma_set_page_prot`	Sets the VMA's page-protection properties.

I observed both the new-VMA path and a VMA merging path during this lab, showing how `do_mmap` handled different mapping requests.

8. The Explicit `mmap()` Request: One Complete ftrace Example

For my final analysis, I extracted the **last complete `mmap` syscall interval** from the combined Function Graph trace using `scripts/14_ftrace_explicit_mmap_evidence.sh`. The extracted file, `notes/ftrace_explicit_mmap_segment.txt`, contains 66 lines from one execution.

The entry event was:

```
sys_mmap(addr: 0, len: 1000, prot: 3,
         flags: 22, fd: ffffffff, off: 0)
```

Here the values are displayed in hexadecimal:

- `len = 0x1000` means **4,096 bytes**.
- `prot = 0x3` means **read and write** permissions (`PROT_READ | PROT_WRITE`).
- `flags = 0x22` means **private, anonymous** mapping (`MAP_PRIVATE | MAP_ANONYMOUS`).
- `fd = -1` is the expected file descriptor value for an anonymous mapping.
- `off = 0` is the mapping offset.

Immediately after the entry event, the trace showed `do_mmap` selecting a virtual address and processing the requested mapping:

```
do_mmap
  -> __get_unmapped_area
    -> mm_get_unmapped_area_vmflags
      -> arch_get_unmapped_area_topdown
        -> vm_unmapped_area
    -> security_mmap_addr
  -> mmap_region
    -> __mmap_region
      -> __mmap_prepare
      -> vma_merge_new_range
        -> vma_expand
          -> commit_merge
      -> vm_stat_account
      -> vma_set_page_prot
```

In this particular call, the kernel used `vma_expand` and `commit_merge`, showing that it expanded and merged a compatible existing VMA.

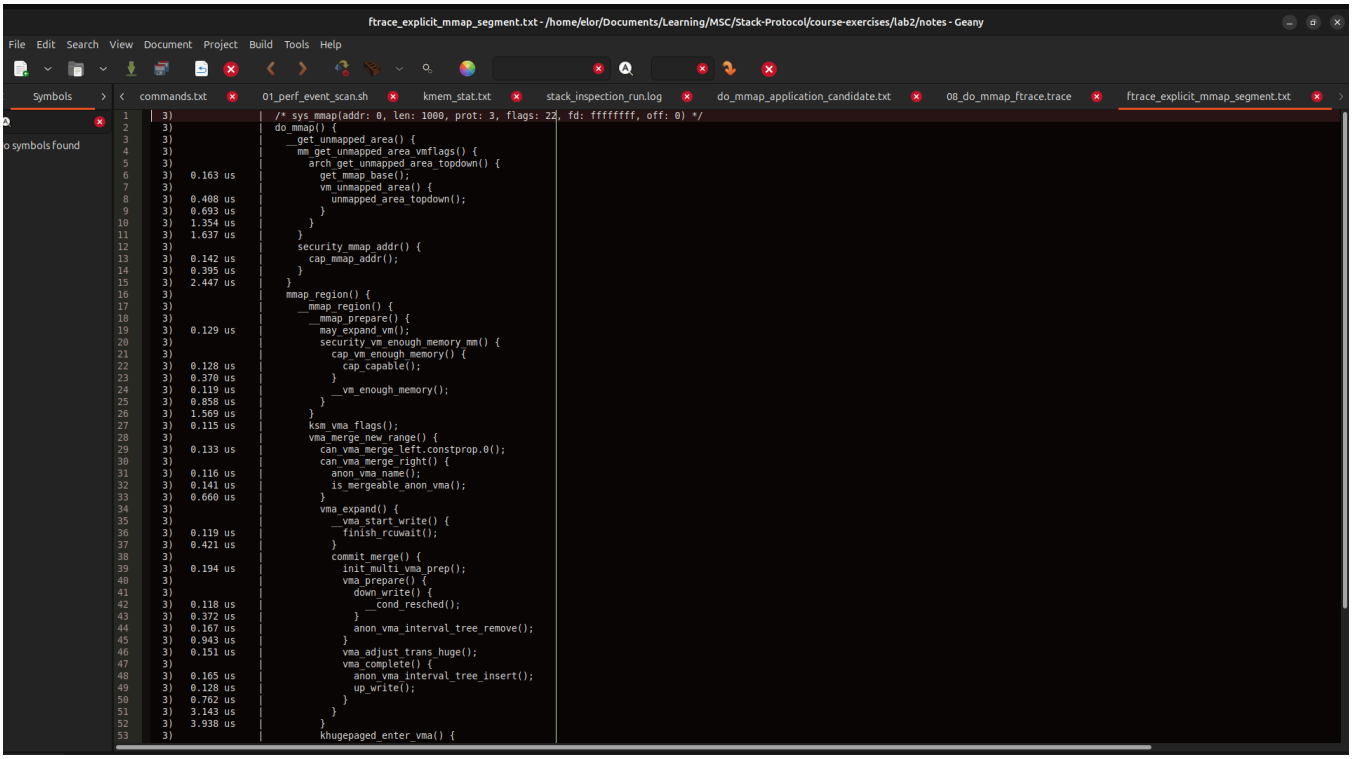


Figure 5 — Explicit mmap request and nested kernel functions. The entry requests 0x1000 bytes with read/write permissions and private anonymous flags; the graph continues through address selection and VMA merging.

I measured the following durations in this one Function Graph:

Function	Recorded duration
`do_mmap`	**12.635 µs**
`__get_unmapped_area`	2.447 µs
`mmap_region`	9.743 µs
`__mmap_region`	9.484 µs
`vma_merge_new_range`	5.885 µs
`vma_expand`	3.938 µs
`commit_merge`	3.143 µs

These are **inclusive, nested durations** from one traced call: each parent duration includes the time spent in its child functions. I used them to describe the work within this specific mapping operation under tracing.

The trace ended with:

```
sys_mmap -> 0x73006ab99000
```

The output of the **same program execution** was:

```
Memory mapped at address: 0x73006ab99000
First byte of memory: A
```

The matching address is direct evidence that the syscall, the nested `do_mmap` calls, and the result printed by the application belong to the same mapping operation.

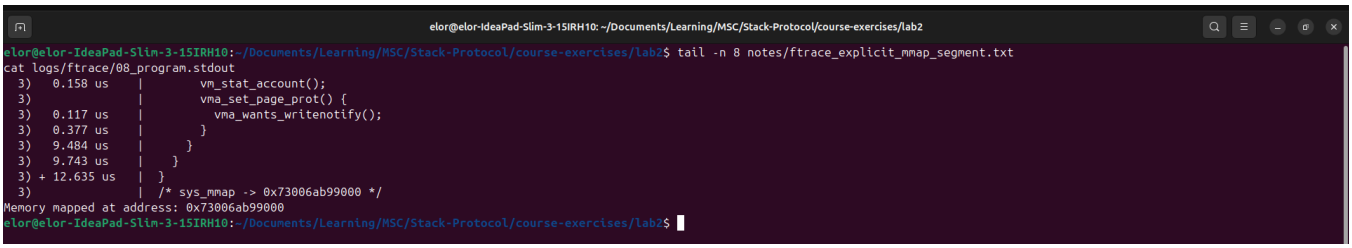


Figure 6 — Matching syscall return and program output. The end of the graph reports 12.635 microseconds for do_mmap and returns 0x73006ab99000, matching the address printed by the same program execution.

9. Checking the ftrace Syscall Event Counts

I initially obtained zero counts for `sys_enter_mmap` and `sys_exit_mmap` in an analysis script. I investigated this using a PID-filtered trace and a second trace without a PID filter.

I found that the events had **actually been recorded**. The problem was the text-search pattern: the ftrace output displayed the tracepoints as `sys_mmap(...)` for entry and `sys_mmap -> ...` for exit, rather than as `sys_enter_mmap:` and `sys_exit_mmap:`.

After correcting the search pattern in `scripts/13_ftrace_correct_counts.sh`, I obtained:

ftrace recording	mmap entries	mmap exits
PID-filtered syscall trace	9	9
Unfiltered syscall trace	178	178
Combined Function Graph trace	9	9

The 178 unfiltered syscall pairs cover the test application and other processes. The nine entry/exit pairs in the PID-filtered trace agreed with the nine `program` pairs observed in Perf.

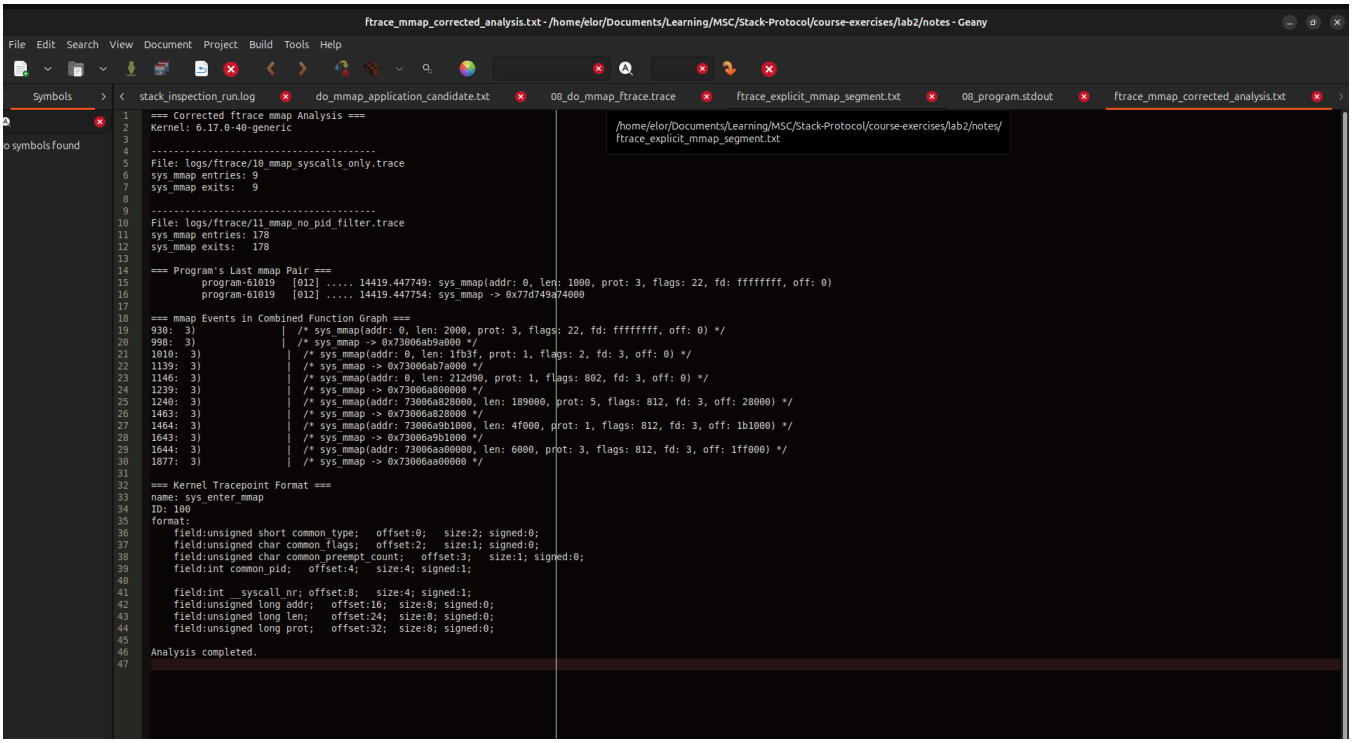


Figure 7 — Corrected ftrace syscall counts. The analysis identifies nine entry/exit pairs in the PID-filtered recording and 178 pairs in the unfiltered recording, alongside syscall records from the combined Function Graph trace.

This troubleshooting step taught me to inspect the actual trace data before interpreting a zero count from a search script.

10. Comparing Perf and ftrace

Both tools helped me understand the same memory operation, but from different directions.

Perf was useful for listing events, counting them, recording samples, and reconstructing **how execution reached `do_mmap`**. Its call stack showed the path from `main` through the C library and the system call entry functions.

ftrace was useful for reconstructing **what happened after `do_mmap` was called**. Its Function Graph showed nested functions, the VMA expansion/merging path, and individual call durations. In the combined recording, it also linked the syscall arguments, kernel functions, and return address in one continuous trace.

I found the two tools complementary: Perf explained the callers of `do_mmap`, and ftrace explained its internal work.